TECHNICAL REFERENCE

Developer Documentation

Tuntaskilat

Architecture, Data Model, Cloud Functions API, Security Rules, and Business Logic — a complete reference for the engineering team

PT Tuntas Kilat Group

Internal Engineering Reference

July 2026

TABLE OF CONTENTS

Contents

TECHNICAL DOCUMENTATION

Tuntaskilat
Developer Reference

On-demand home cleaning service platform — 3 Flutter client apps (Customer, Worker, Admin) on a shared Firebase backend. This reference covers the data model, Cloud Functions API, security rules, and business logic that power it.

Get Started → End-user Manual

3	22	20+	1
Client Apps	Cloud Functions	Firestore Collections	Shared Package (tk_core)

Explore the docs

Getting Started
Clone the repo, install dependencies, run each app locally.

Data Model
Full Firestore schema — every collection, field, and enum.

Cloud Functions

All 22 server-side functions: triggers, inputs, outputs.

Security Rules

Read/write access matrix for every collection.

AI Features

Multi-provider AI customer service & business analyst.

Payments

Static gateway + Xendit dynamic invoicing/webhook.

Tech Stack

stack.yaml

```yaml
client:
  framework: Flutter 3.41
  state: Riverpod 2.6
  language: Dart 3.11
  apps: [pelanggan, kru, admin]
  shared_package: packages/tk_core

backend:
  platform: Firebase
  functions: Node.js 20 (2nd Gen), TypeScript
  region: asia-southeast2
  database: Cloud Firestore (NoSQL, document-based)
  auth: Firebase Authentication (email/password + Google)
  storage: Cloud Storage
  messaging: Firebase Cloud Messaging (FCM)

integrations:
  ai: [Gemini, OpenAI, Anthropic]  # multi-provider, admin-configurable
  payments: [transfer_bank, qris, tunai, xendit]
```

```
  whatsapp: Fonnte
```

INTRODUCTION

Getting Started

Clone the monorepo, install dependencies, and get all three apps running locally.

Prerequisites

You'll need the following tools installed:

```
Flutter SDK    3.41.x (stable channel)
Dart SDK       3.11.x
Node.js        20.x
Firebase CLI   latest  (npm i -g firebase-tools)
A Firebase project with Firestore, Auth, Storage, Functions, FCM enabled
```

Repository Structure

This is a Flutter pub workspace monorepo — one Git repository, multiple apps sharing a common package:

```
tkapps/
├── apps/
│   ├── pelanggan/     # Customer app (Android)
│   ├── kru/           # Worker app (Android)
│   └── admin/         # Admin panel (Flutter Web)
├── packages/
│   └── tk_core/       # Shared models, services, theme, widgets
├── firebase/
```

```
│   ├── functions/       # Cloud Functions (TypeScript)
│   ├── firestore.rules
│   └── storage.rules
├── docs-site/
│   ├── manualbook/      # This site's sibling — end-user manual
│   └── documentation/   # You are here
├── firebase.json        # Hosting targets, functions, rules
└── pubspec.yaml          # Workspace root
```

Local Setup

```
git clone https://github.com/nabhanyuzqi1/tuntaskilat.git
cd tuntaskilat

# Flutter workspace — installs deps for all 3 apps + tk_core at once
flutter pub get

# Cloud Functions
cd firebase/functions
npm install
cd ../..

# Connect to your own Firebase project (or use the existing one)
firebase use --add
```

ⓘ Each app needs its own `firebase_options.dart`, generated via `flutterfire configure` — run it once per app (`apps/pelanggan`, `apps/kru`, `apps/admin`).

Running Each App

```
# Customer app (Android emulator/device)
cd apps/pelanggan && flutter run

# Worker app
cd apps/kru && flutter run

# Admin panel (web)
cd apps/admin && flutter run -d chrome

# Cloud Functions emulator
cd firebase && firebase emulators:start --only firestore,functions
```

Testing

```
# Unit + widget tests (business logic, pricing, security scenarios)
flutter test packages/tk_core
flutter test apps/admin

# Static analysis across the workspace
flutter analyze packages/tk_core apps/admin apps/pelanggan apps/kru

# Cloud Functions type-check
cd firebase/functions && npx tsc -p .
```

Next
Architecture

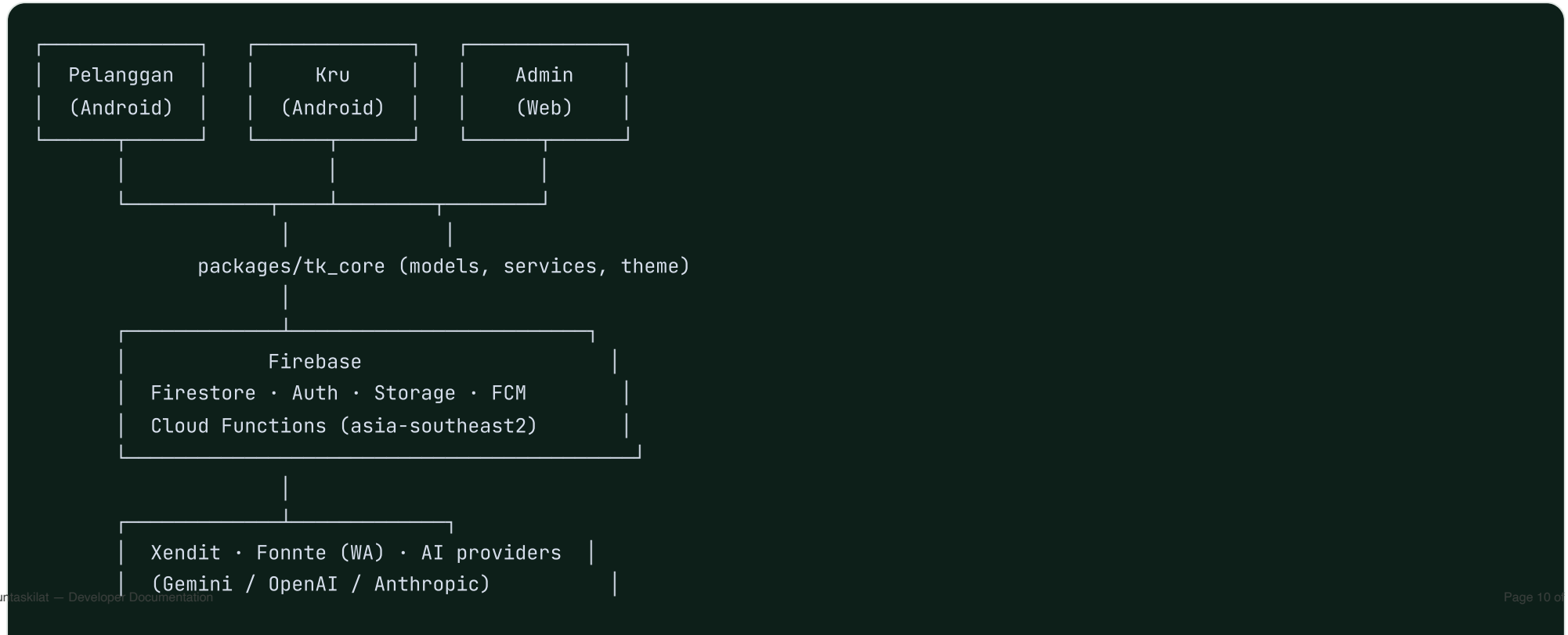
INTRODUCTION

Architecture

A monorepo of 3 Flutter clients sharing one package, backed by Firebase — Firestore as the source of truth, Cloud Functions as the money-critical safety net.

System Overview

Every client writes optimistically via a Firestore transaction (client-side pricing/locking), and every write of consequence is re-validated by a Cloud Function running with the Admin SDK — which bypasses Security Rules and is treated as the single source of truth for money. The client is a preview; the server is authoritative.



Client Applications

App	Platform	Users	Purpose
apps/pelanggan	Android	Customers	Browse services, book, pay, track, review, chat, AI CS.
apps/kru	Android	Field crew	Accept jobs, navigate, report work, manage cash deposits.
apps/admin	Flutter Web	Staff/owner	Manage orders, catalog, crew, vouchers, commission, operations.

Shared Package (tk_core)

`packages/tk_core` holds everything that must stay consistent across apps: Firestore models & enums, the `FirestoreService` facade (all reads/writes/transactions), auth flow, theme tokens (colors, typography, radius), and shared widgets. No app talks to Firestore directly — everything routes through `tk_core`.

Backend Services

Service	Role
Cloud Firestore	Document-based NoSQL database — the system of record for orders, users, payments, etc.
Firebase Auth	Email/password + Google sign-in. Role (pelanggan/kru/admin) stored on the users document, not in Auth claims.
Cloud Storage	Payment proof uploads, work-report photos, profile photos, service images.
Cloud Functions	22 functions (Node.js 20, TypeScript, region asia-southeast2) — triggers, scheduled jobs, and callables. See Functions reference.

Service	Role
FCM	Push notifications for order status changes, chat messages, assignment reminders, broadcasts.

Auth & Roles

A single `UserRole` enum (`pelanggan` | `kru` | `admin`) gates which app an account can log into. `AuthService.signIn()` checks the role against the app performing the login and signs the session out immediately on a mismatch (a pelanggan account cannot log into the Kru portal, etc.) — this check exists both client-side and mirrored in Security Rules.

Role	Self-registration?	How the account is created
pelanggan	Yes	registerPelanggan() — email/password or Google sign-in, self-service.
kru	No	Admin only, via A5 → AkunKruService.buatAkunKru() (uses a secondary Firebase App instance so the admin's own session is never disturbed).
admin	No	Existing admin only, via the buatAdmin callable Cloud Function (admin.auth().createUser() server-side).

ⓘ Admin accounts can additionally require TOTP 2FA (RFC 6238) at login — secret stored in `admin2fa/{uid}`, owner-only read/write.

Atomic Locking Pattern

Booking a time slot is a classic race condition — two customers could both read "slot open" and both write a booking. Tuntaskilat solves this without a dedicated locking service, using a Firestore transaction against a capacity counter:

simplified booking transaction

```
await db.runTransaction(async (tx) ⇒ {
```

```
  const slotSnap = await tx.get(slotRef)
  const { terisi = 0, kapasitas } = slotSnap.data() ?? {}

  if (terisi ≥ kapasitas) throw new JadwalPenuhException()

  // Firestore rule enforces this write can ONLY increment by exactly 1
  tx.set(slotRef, { terisi: terisi + 1, kapasitas }, { merge: true })
  tx.set(orderRef, order.toMap())
})
```

`kapasitas` (capacity) is not a fixed number — it's recomputed automatically whenever crew data changes (see `recomputeKapasitasLayanan()` in the Functions reference), so booking availability always reflects how many eligible crew are actually active for that service.

Previous
Getting Started

Next
Data Model

API REFERENCE

Data Model

Every Firestore collection Tuntaskilat writes to — field names, types, and enum values, sourced directly from packages/tk_core/lib/models/.

Model source files live in `packages/tk_core/lib/models/` — one Dart class per collection, each with typed `fromMap()` / `toMap()` so every read/write goes through the same shape.

users/{userId}

`UserModel`

One document per account, docId = Firebase Auth UID. Shared shape across all 3 roles.

nama	`string`	
email	`string`	
noTelepon	`string`	
alamat	`string`	
role	`UserRole`	
fotoUrl	`string`	
nonaktif	`bool`	company-deactivated account flag
kodeReferal	`string`	deterministic 6-char base32 hash of uid, prefixed TK
referredBy	`string`	referral code entered at signup

Enums

UserRole	enum	pelanggan | kru | admin

Subcollection users/{uid}/alamat: { label, alamat, lokasi: GeoPoint } — saved addresses.

services/{serviceId}

`ServiceModel`

Catalog of bookable cleaning services with dynamic pricing.

namaLayanan	string	
deskripsi	string	
harga	num	base / "starting from" price
satuan	string	per jam | per ruangan | per m2
aktif	bool	
kategori	string	rumput | home_cleaning | umum
gambarUrl	string	
tipeHarga	TipeHarga	
tiers	TarifTier[]	{ id, nama, hargaPerM2 } — for per_luas pricing
paketOpsi	PaketOpsi[]	{ id, nama, jumlahPetugas, durasi[], hargaTambahJam, spesifikasi }
addOns	AddOn[]	{ id, nama, harga }
jumlahKru	num	capacity — maintained by Cloud Function, never written by admin edits

Enums

TipeHarga	enum	per_luas | paket | mulai_dari

orders/{orderId}

OrderModel

The central document — every transaction and its full lifecycle.

userId	string	
serviceId	string	
cleanerId	string	lead crew member, "" = unassigned
tanggalPesan / jadwal	Timestamp	
totalHarga / subtotal / potongan	num	totalHarga = subtotal − potongan
status	OrderStatus	
penugasan	Penugasan[]	{ cleanerId, nama, peran, sudahKonfirmasi, diterima, fotoUrl }
kruIds	string[]	for array-contains queries
metodePembayaran	MetodeBayar	
voucherKode / rincian	string / BarisRincian[]	
fotoSebelum / fotoSesudah	string[]	work-report photos
slotId	string	written manually, not in toMap() — links to the capacity slot
kasTunaiDibukukan	bool	idempotency guard for cash-commission booking

Enums

OrderStatus	enum	dibuat → menunggu_pembayaran → menunggu_verifikasi → (ditolak ↩ | terverifikasi) → menunggu_penugasan → ditugaskan → dalam_perjalanan → diproses → selesai → dinilai, + dibatalkan
MetodeBayar	enum	transfer_bank | qris | tunai

payments/{paymentId}

PaymentModel

Payment proof + verification status. Xendit adds ad-hoc gateway/invoiceId/invoiceUrl fields not in the Dart model.

orderId / userId	string	
metode	MetodeBayar	
jumlah	num	
buktiBayar	string?	Storage URL
statusBayar	StatusBayar	

Enums

StatusBayar	enum	menunggu | terverifikasi | ditolak

payouts/{orderId}_{cleanerId}

PayoutModel

Crew wage ledger — immutable to clients once created. Money is never trusted from the client after this point.

orderId / cleanerId / namaKru	string	

peran	`PeranKru`
jumlah	`num`
status	`StatusPayout`

Enums

PeranKru	`enum`	worker (weight 1.0) | helper (weight 0.6)
StatusPayout	`enum`	pending | dibayar | ditahan

kru/{cleanerId}

`KruModel`

Crew profile, docId = Firebase Auth UID.

nama / noTelepon / fotoUrl	`string`	
statusKetersediaan	`bool`	online/offline toggle
posisi	`GeoPoint?`	live location, streamed while dalam_perjalanan
rataRating / jumlahUlasan	`num`	server-authoritative, recomputed on every review
fcmTokens	`string[]`	
keahlian	`string[]`	serviceIds — empty array = generalist (eligible for all services)
tipe	`KruTipe`	
status	`StatusKru`	only aktif is assignable
rekening	`RekeningKru`	{ jenis: bank|ewallet, penyedia, nomor, atasNama }

Enums

KruTipe	enum	kru | mitra | vendor
StatusKru	enum	aktif | nonaktif | diberhentikan

kasKru/{cleanerId}

KasKru

Cash-deposit ledger — how much commission from cash-paid jobs a crew member is still holding.

namaKru	string	
saldoTunai	int	≥ 0. Invariant: saldoTunai == totalMasuk − totalSetor
batasNunggak	int	default 200000 (Rp) — arrears threshold
totalMasuk / totalSetor	int	lifetime accumulated in / deposited out

setoran/{setoranId}: { cleanerId, namaKru, jumlah, diterimaOleh, waktu, catatan } — immutable deposit history.

vouchers/{kode}

VoucherModel

docId = uppercase code.

tipe	TipeVoucher	
nilai	num	percent or fixed amount depending on tipe
minBelanja / maxPotongan	num	maxPotongan 0 = unbounded
kuota / terpakai	int	kuota 0 = unlimited
berlakuHingga	string?	ISO date

khususPenggunaBaru `bool`	
sekaliPerNomor `bool`	default true — locks claim per phone number, not per uid

Enums

TipeVoucher `enum`	persen | nominal

voucherUsages/{kode}__{telepon}: anti-abuse lock, immutable to the user once created.

reviews/{reviewId}

`ReviewModel`

Public — every review re-aggregates the rated crew member's kru.rataRating server-side (onReviewCreate).

orderId / userId / cleanerId `string`	
penilaian `num`	1–5
komentar / namaPelanggan `string`	

notifications/{id}

`NotificationModel`

In-app notification feed, mirrors FCM pushes.

userId `string`	recipient
judul / pesan `string`	
dibaca `bool`	
orderId `string?`	deep-link target

disputes/{disputeId}

`DisputeModel`

Appeals over work quality or payout amount.

orderId / pengajuId / pengajuNama	`string`
pengaju	`PengajuBanding`
status	`StatusBanding`
alasan / catatanAdmin	`string`
buktiUrl	`string?`

Enums

PengajuBanding	`enum`	kru | pelanggan
StatusBanding	`enum`	diajukan | diterima | ditolak

banners/{id} · broadcasts/{id}

`BannerModel · (none)`

Two unrelated concepts despite the similar name — see the AI/Notifications pages for how each is used.

banners.judul / subjudul / badge / gambarUrl / isi / tautan / urutan / aktif	– persistent, admin-curated promo content shown on the Beranda
broadcasts.judul / pesan / terkirim / tanpaToken / selesaiPada	– one-shot marketing push blast to all pelanggan, no dedicated model class

settings/{docId}

`various`

Runtime configuration, admin-controlled. See individual docIds below.

komisi	`KonfigKomisi`	global commission % + per-service overrides
pembayaran	`KonfigPembayaran`	which payment methods are active/static/dynamic
app	`KonfigApp`	maintenance mode / forced update — publicly readable
ai	`KonfigAi`	provider + API keys for AI features — admin-only, secret
referral	`{ nominal, minBelanja, masaBerlakuHari }`	referral reward configuration

referralClaims/{telepon}

`(none)`

One document per phone number, ever — the anti-abuse lock for the referral program.

telepon / buyerId / referrerId	`string`
kodeReferal / voucherKode	`string`

admin2fa/{uid}

`(none)`

TOTP secret for admin 2FA. Owner-only read/write.

secret	`string`
aktif	`bool`

slots/{slotId}

`(none)`

Capacity counter backing the atomic-locking booking mechanism.

serviceId / jadwal	`string / Timestamp`	
terisi / kapasitas	`num`	terisi can only be incremented by exactly +1 per client write (rule-enforced); decrements are admin-SDK only

Previous
Architecture

Next
Cloud Functions

API REFERENCE

Cloud Functions

22 server-side functions — all running in asia-southeast2 as a safety-net layer that mirrors and validates client-side transactions.

ⓘ Every function shares one Firestore admin instance and the `REGION = "asia-southeast2"` constant, defined in `firebase/functions/src/index.ts`.

Orders & Pricing Integrity

onOrderCreate `onDocumentCreated` orders/{orderId}

Re-reads the service document and recomputes the correct price server-side, correcting the order in place if the client-submitted totalHarga/subtotal/potongan drift by more than Rp1.

> Anti price-tampering: the client computes a preview price for UX, but this function is what actually decides what the customer owes.

validatePayment `onDocumentCreated` payments/{paymentId}

Confirms payment.jumlah equals the referenced order's totalHarga (tolerance Rp1); corrects it if mismatched.

onOrderFinalize `onDocumentUpdated` orders/{orderId}

Fires on transition into status: 'selesai'. Computes crew payouts (weighted split, helper ratio 0.6) and, for cash orders, books the platform commission into the lead crew member's kasKru ledger.

> Idempotent via the kasTunaiDibukukan flag on the order document — cash commission is only ever booked once.

onOrderAssigned `onDocumentUpdated` `orders/{orderId}`

Fires on transition into 'ditugaskan'. Sends FCM push to every assigned crew member's fcmTokens, cleaning up invalid tokens.

onOrderCancelled `onDocumentUpdated` `orders/{orderId}`

Fires on transition into 'dibatalkan'. Decrements the booking slot's terisi counter (or deletes the slot doc) — the only permitted decrement path, since clients can only increment.

pushStatusPelanggan `onDocumentUpdated` `orders/{orderId}`

Pushes FCM to the customer on every meaningful status transition, with data.tipe='status' for deep-linking into the tracking screen.

autoCancelTanpaKru `onSchedule` `every 60 minutes (Asia/Makassar)`

Cancels orders stuck in 'terverifikasi'/'menunggu_penugasan' for more than 24 hours with no crew assigned, notifying the customer.

Chat & Notifications

onChatMessageCreate `onDocumentCreated` `orders/{orderId}/messages/{messageId}`

Determines recipients (pelanggan → all kruIds; kru → userId), writes a notification doc, and pushes FCM.

reminderKru `onSchedule` `every 30 minutes (Asia/Makassar)`

Finds 'ditugaskan' orders whose jadwal is 90–150 minutes away and sends a reminder push to assigned crew.

onBroadcastCreate `onDocumentCreated` `broadcasts/{id}`

Mass-pushes an FCM notification to every user with role=='pelanggan', then writes delivery stats back onto the broadcast doc.

waNotifOrder `onDocumentUpdated` `orders/{orderId}`

Sends a WhatsApp message via Fonnte for terverifikasi/ditugaskan/selesai status transitions. No-op if FONNTE_TOKEN is unset.

Reviews & Capacity

onReviewCreate `onDocumentCreated` `reviews/{reviewId}`

Re-aggregates ALL reviews for the rated cleanerId and overwrites kru/{cleanerId}.rataRating / jumlahUlasan — server-authoritative, never client-computed.

onKruDitulis `onDocumentWritten` `kru/{cleanerId}`

Any create/update/delete of a crew doc triggers recomputeKapasitasLayanan().

onLayananDibuat `onDocumentCreated` `services/{serviceId}`

A new service triggers recomputeKapasitasLayanan() so jumlahKru is populated immediately.

backfillKapasitasKru `onCall` `admin-only`

Manually forces a full recompute of service capacities — used after bulk keahlian edits.

Admin & Referral

buatAdmin `onCall` `admin-only`

Input { email, password, nama }. Creates a Firebase Auth user + users doc (role: admin) without disturbing the caller's session. Returns { uid }.

setNonaktifAdmin `onCall` `admin-only`

Input { uid, nonaktif }. Disables/enables the target Auth account. Cannot target self (failed-precondition).

rewardReferral `onDocumentUpdated` `orders/{orderId}`

On a buyer's first completed order, mints a reward voucher for the referrer, guarded by a transaction on referralClaims/{phone} — one claim per phone number, ever.

Payments (Xendit)

xenditWebhook `onRequest` `HTTP endpoint`

Validates x-callback-token, then on status PAID/SETTLED marks the matching payment & order 'terverifikasi'. Returns 501 if XENDIT_CALLBACK_TOKEN is unset.

buatTagihanXendit `onCall` `auth required`

Input { orderId, amount }. Verifies order ownership, calls the Xendit Invoices API, persists invoiceUrl/invoiceId on the payment doc. Requires XENDIT_SECRET_KEY.

AI

csAi `onCall` `auth required`

Input { pertanyaan, orderId? }. Multi-provider chat (Gemini/OpenAI/Anthropic) grounded in the live service catalog + the caller's own order status. Returns { jawaban }.

analisaBisnisAi `onCall` `admin-only`

Aggregates the trailing 30 days of orders (numerically, server-side) and asks the configured LLM for a concise business summary + action suggestions. Returns { ringkasan }.

Shared Helpers

Two utilities worth knowing when reading the source:

```
// Reads settings/komisi — perLayanan[serviceId] override, else global, clamped 0-100.
async function komisiPersenUntuk(serviceId: string): Promise<number>

// Recomputes services.jumlahKru = count of active kru (status undefined/null/'aktif')
// whose keahlian is empty (generalist) or includes the serviceId. Drives the
// capacity-based booking slot mechanism.
async function recomputeKapasitasLayanan(): Promise<void>
```

Previous
Data Model

Next
Security Rules

API REFERENCE

Security Rules

firebase/firestore.rules — the second line of defense alongside Cloud Functions.
Read/write access per collection.

Helper Functions

```
function isSignedIn() { return request.auth != null; }
function role() { return get(/databases/$(db)/documents/users/$(request.auth.uid)).data.role; }
function isOwner(uid) { return isSignedIn() && request.auth.uid == uid; }
function isAdmin() { return isSignedIn() && role() == 'admin'; }
```

Access Control Matrix

Collection	Read	Create	Update	Delete
`users/{userId}`	owner or admin	owner only	owner or admin	admin
`services`	public	—	admin	admin
`slots/{slotId}`	any signed-in	signed-in, terisi must be exactly +1	same rule, or admin	admin
`orders/{orderId}`	participant or admin	signed-in owner; price fields must match services	admin, or participant limited to status/photos/cancellation	admin

Collection	Read	Create	Update	Delete
`orders/{id}/messages`	participant or admin	participant or admin	—	—
`kru/{cleanerId}`	public	admin	owner/admin, or signed-in diff-only on rating fields	admin
`settings/ai`	admin only	admin	admin	admin
`settings/{other}`	signed-in, or public if docId=='app'	admin	admin	admin
`payments`	owner or admin	signed-in owner	admin only	—
`reviews`	public	signed-in owner	admin	admin
`notifications`	owner	any signed-in (ideally server-written)	—	—
`vouchers/{kode}`	signed-in	admin	admin, or diff-only terpakai == old+1	admin
`voucherUsages`	get: any signed-in; list: admin/owner	signed-in owner	admin	admin
`broadcasts / banners`	admin / public	admin	admin	admin
`admin2fa/{uid}`	owner	owner	owner	owner
`payouts`	admin or owning cleaner	admin or signed-in (in completion tx)	admin	admin
`kasKru`	admin or owning cleaner	—	admin only	admin
`setoran`	admin or owning cleaner	admin	admin	admin
`referralClaims`	admin or any signed-in	—	admin only	admin

Collection	Read	Create	Update	Delete
`disputes`	admin or the filer	signed-in filer, status must be diajukan	admin	admin

Key Invariants

> ⚠ Order money/ownership fields are immutable to non-admins once created — a client can only ever touch `['status', 'fotoSebelum', 'fotoSesudah', 'penugasan']` on an existing order, and cancellation is only permitted from pre-assignment statuses.

Other write-shape constraints enforced at the rules layer:

- Slot `terisi` can only be incremented by exactly +1 per client write — decrements are admin-SDK only (the atomic-locking mechanism).
- Voucher `terpakai` can only increment by exactly 1 per write.
- `kru` rating fields (`rataRating`, `jumlahUlasan`) can only move via a controlled delta matching a genuine review submission.
- `kasKru` balances are never client-writable — only Cloud Functions (admin SDK, which bypasses rules entirely) or admin-authenticated writes can change them.

Previous
Cloud Functions

Next
Commission & Payouts

BUSINESS LOGIC

Commission & Payouts

The exact algorithms behind money in Tuntaskilat — every formula here is enforced server-side, never trusted from the client.

Wage Splitting — bagiUpah()

`packages/tk_core/lib/models/wage.dart` — splits a completed order's total between platform commission and crew, weighted by role.

```
totalInt   = round(total)
komisi     = round(totalInt × komisiPersen / 100)     // platform commission
pool       = totalInt - komisi                        // remaining for crew
totalBobot = Σ peran.bobot for each crew member         // worker=1.0, helper=0.6

for each crew member p:
    bagian[p] = floor(pool × p.peran.bobot / totalBobot)

sisa = pool - Σ bagian                                  // rounding remainder
bagian[crew.first] += sisa                               // remainder → lead/first member

# Guarantee: komisi + Σ bagian == total, exactly. No Rupiah lost or created.
```

⚠ The server-side `onOrderFinalize` Cloud Function reimplements the same weighted split inline, but rounds each share *individually* rather than floor + remainder-to-first — so its per-share numbers can differ from the client's `bagiUpah()` preview by ±1 for the same input. The client value is only a preview; the Cloud Function is authoritative for actual payouts.

Commission Config — KonfigKomisi

```
persenUntuk(serviceId) = clamp(perLayanan[serviceId] ?? komisiPersen, 0, 100)
```

Mirrored server-side as `komisiPersenUntuk(serviceId)`, reading `settings/komisi`. Global default is 20%.

Cash Deposit Ledger — KasKru

For cash-paid orders, the crew member collects the full amount from the customer in person — but the platform's commission share is still owed. That amount is tracked as a debt (`saldoTunai`) the crew must periodically settle with the office.

```
saldoTunai == totalMasuk - totalSetor        // audit invariant
bolehTerimaTunai == saldoTunai ⩽ batasNunggak   // default batasNunggak = Rp200,000
```

`saldoTunai` increases exactly once per cash order, booked by `onOrderFinalize` (guarded by the `kasTunaiDibukukan` flag) — no client path can write it directly (Security Rules restrict `kasKru` writes to admin). It decreases only when an admin records a deposit via `terimaSetoran()`, which runs inside a transaction that rejects any deposit amount exceeding the current balance.

Voucher Discount Formula

```
if !aktif                          → reject 'nonaktif'
if now > berlakuHingga             → reject 'kadaluarsa'
if sisaKuota ⩽ 0                   → reject 'kuotaHabis'   // sisaKuota = kuota⩽0 ? ∞ : kuota-terpakai
if subtotal < minBelanja           → reject 'minimalBelanja'

potongan = tipe=='persen' ? subtotal × nilai / 100 : nilai
if tipe=='persen' && maxPotongan>0 && potongan>maxPotongan
    potongan = maxPotongan
if potongan > subtotal
    potongan = subtotal
```

The server-side `potonganFormulaVoucher()` applies the same nominal-cap math WITHOUT the kuota/expiry/aktif gates (those are enforced client-side inside the booking transaction) — its purpose is purely to cap the claimed amount: `potonganBenar = min(potonganKlien, formulaMax)`, preventing a client from claiming more discount than the voucher's own math allows.

Voucher Anti-Abuse

Two independent guards inside `buatPesananLengkap()`:

New-user-only vouchers

Checked *outside* the transaction (a query for any prior order by `userId`) — if the user has ordered before, throws `VoucherException(hanyaPenggunaBaru)`.

Per-phone-number lock

Doc ID `voucherUsages/{kode}__{normalizedPhone}` (phone normalized: `0xxxx → 62xxxx`, non-digits stripped). Inside the transaction, if that doc already exists → reject. This is keyed on **phone number, not uid** — specifically to prevent the "make a new account, reuse the phone number" abuse vector. `voucher.terpakai` is incremented by exactly 1 in the same transaction, matching the Security Rules constraint.

Capacity-Based Slots

Replaces a legacy boolean "taken" lock with a live counter. `services.jumlahKru` — maintained by `recomputeKapasitasLayanan()`, triggered by `onKruDitulis` / `onLayananDibuat` / `backfillKapasitasKru` — is the count of active crew eligible for that service (empty `keahlian` = generalist, or `keahlian` includes the serviceId). This becomes the hourly booking capacity.

```
kapasitas = services.jumlahKru  (fallback 1 if field absent)
terisi ≥ kapasitas   → JadwalPenuhException
kapasitas ≤ 0        → TidakAdaKruException
```

A service with zero eligible crew is fully locked for booking. Cancellation (`onOrderCancelled`) decrements `terisi` via the admin SDK — the only permitted decrement path.

Cash Arrears Guard

```
if (order.tunai && !await bolehTerimaTunai(lead.cleanerId)) {
  throw KruNunggakException(lead.nama);
}
```

Before assigning a cash-payment order to a lead crew member, `tugaskanKruMulti()` checks `kasKru/{lead}.bolehTerimaTunai`. A crew member who has accumulated more than the arrears threshold in un-deposited cash commission cannot be assigned new cash-paying jobs until they settle up.

Previous
Security Rules

Next
Payments & Gateway

INTEGRATIONS

Payments & Gateway

Payment methods are pluggable — a manual verification flow ships by default, with Xendit available as an opt-in dynamic gateway.

Gateway Abstraction

`packages/tk_core/lib/services/payment_gateway.dart` defines a `PaymentGateway` interface to avoid vendor lock-in:

```
abstract class PaymentGateway {
  Future<InstruksiBayar> buatInstruksi({
    required String orderId,
    required int jumlah,
    required MetodeBayar metode,
  });
  String get nama;
}

class StaticGateway implements PaymentGateway { ... }  // active by default
class XenditGateway implements PaymentGateway { ... }   // opt-in
```

Static vs. Dynamic

	StaticGateway	XenditGateway
Instructions	Fixed bank account / QRIS image from settings/pembayaran	Per-order dynamic VA / QRIS from Xendit
Verification	Manual — customer uploads proof, admin approves/rejects	Automatic — settled via webhook
Requires	Nothing extra (default MVP flow)	XENDIT_SECRET_KEY + XENDIT_CALLBACK_TOKEN env vars
Status if unconfigured	Always available	buatTagihanXendit throws failed-precondition; xenditWebhook returns HTTP 501

Both paths land on the same `payments` collection and, once `terverifikasi`, feed into the same commission / payout calculation — they're layered, not competing systems.

Xendit Flow

```
1. Customer selects a Xendit-backed method at checkout
2. App calls buatTagihanXendit({ orderId, amount })
     → verifies caller owns the order
     → POST https://api.xendit.co/v2/invoices  (Basic auth from XENDIT_SECRET_KEY)
     → persists { gateway, invoiceId, invoiceUrl } on the payment doc
     → returns invoiceUrl for the app to open
3. Customer completes payment on Xendit's hosted page
4. Xendit calls xenditWebhook with x-callback-token + { external_id: orderId, status }
     → validates token (401 if wrong)
     → on PAID/SETTLED: payments.statusBayar = 'terverifikasi', orders.status = 'terverifikasi'
```


Commission vs. Payment Config — Two Different Things

It's easy to confuse `settings/komisi` and `settings/pembayaran` because both live under "money settings" in the admin panel — but they govern orthogonal concerns:

Setting	Controls
`settings/komisi`	The platform's revenue-share percentage taken from each completed order, used by bagiUpah/onOrderFinalize to split money between platform and crew.
`settings/pembayaran`	Which customer-facing payment methods are offered and whether each is statis (manual) or dinamis (Xendit).

A `dinamis` payment method still goes through the same commission calculation once the order completes — the two systems are layered, not competing.

Previous
Commission & Payouts

Next
AI Features

INTEGRATIONS

AI Features

Two AI-powered callables — a customer-facing support chat and an admin-facing business analyst — both provider-agnostic.

Multi-Provider Config

`settings/ai` (admin-only, read/write) selects which provider is active and holds its key:

Field	Notes
provider	gemini | openai | anthropic
geminiApiKey / openaiApiKey / anthropicApiKey	per-provider key field; falls back to env vars (GEMINI_API_KEY, etc.) if empty
model	defaults: gemini-2.0-flash, gpt-4o-mini, claude-haiku-4-5
aktif	master on/off switch

⚠ Because keys fall back to environment variables, a deployed env var can silently supply a key even if the admin panel shows none configured — worth checking both when debugging "AI not responding".

A single dispatcher, `panggilAi()`, routes to the correct REST endpoint per provider: Gemini's `generateContent` (key in query string), OpenAI's `/chat/completions`, or Anthropic's `/v1/messages`.

Customer Service AI — csAi

```
// onCall, auth required
Input:  { pertanyaan: string, orderId?: string }
Output: { jawaban: string }
```

The system prompt is grounded in the **live active-service catalog and prices**, fixed operating hours (08.00–19.00 WIB), the 3 payment methods, and cancellation policy. If `orderId` is supplied and belongs to the calling uid, that order's current status is injected as additional context — the assistant only ever sees the caller's own data.

Fallback: if the AI config's key is missing or `aktif` is false, the function throws `failed-precondition` ("Asisten AI belum diaktifkan").

Business Analyst AI — analisaBisnisAi

```
// onCall, admin-only
Output: { ringkasan: string }
```

Purely numeric aggregation is done server-side in TypeScript — the LLM never touches the database directly. Over the trailing 30 days of `orders`, the function computes: total / selesai / dibatalkan counts, omzet (revenue from completed orders), and a per-service order-count breakdown — then hands those numbers to `panggilAi()` with a prompt capped at 6 bullet points plus 2–3 concrete action suggestions.

Guardrails

- The prompt explicitly forbids fabricating data or leaking other customers' PII.
- `analisaBisnisAi` is told not to fabricate beyond the numbers it was given.
- Both functions require the caller to be authenticated; the business analyst additionally requires `role == 'admin'`.

INTEGRATIONS

Notifications

Two independent notification channels — Firebase Cloud Messaging for push, and WhatsApp via Fonnte for a persistent paper trail.

FCM Push Events

Function	Recipient	Trigger
onOrderAssigned	assigned crew	order enters 'ditugaskan'
pushStatusPelanggan	customer	terverifikasi / ditugaskan / dalam_perjalanan / dikerjakan / selesai / ditolak
onChatMessageCreate	the other chat participant	any new message in orders/{id}/messages
reminderKru	assigned crew	every 30 min, if jadwal is 90–150 min away
onBroadcastCreate	all pelanggan with a token	admin creates a broadcasts/{id} doc

ⓘ Invalid/expired tokens are cleaned from `kru.fcmTokens` automatically whenever a push fails with `registration-token-not-registered` or `invalid-argument`.

WhatsApp (Fonnte) — waNotifOrder

A thin abstraction, `WaSender`, with `FonnteSender` as the active implementation — swap the class if you switch WhatsApp providers without touching call sites. Fires on the same order-status transitions as the push notification (terverifikasi / ditugaskan / selesai), sending a formatted text message to the customer's phone. No-op — safely, silently — if `FONNTE_TOKEN` is unset in `firebase/functions/.env`.

In-App Notification Feed

Every push additionally writes a `notifications` document (see Data Model) so the customer/crew has a persistent, scrollable history inside the app — not just a transient system notification. Notifications are deep-linkable via the `orderId` field.

Previous
AI Features

Next
Deployment

OPERATIONS

Deployment

One Firebase project, multiple hosting sites, one Cloud Functions codebase.

Hosting Targets

`firebase.json` declares multiple hosting entries, each mapped to a distinct Firebase Hosting site via `.firebaserc` target aliases:

Target	Public dir	Serves
admin	apps/admin/build/web	Admin panel (Flutter web build)
manualbook	docs-site/manualbook/dist	This end-user manual
documentation	docs-site/documentation/dist	This developer reference

Site IDs must be globally unique across all Firebase projects. If a bare name like `manualbook` is taken, Firebase assigns a project-prefixed fallback (e.g. `tuntaskilat-manualbook`) — check the actual `*.web.app` URL printed after `firebase hosting:sites:create`.

Cloud Functions Env Vars

Optional integrations are gated behind environment variables in `firebase/functions/.env` — every one of them degrades gracefully (no-op, HTTP 501, or a clear `failed-precondition` error) when unset, so partial configuration never breaks the core booking flow.

```
# WhatsApp notifications (waNotifOrder) — no-op if unset
FONNTE_TOKEN=

# Xendit dynamic payments — buatTagihanXendit throws if unset
XENDIT_SECRET_KEY=

# Xendit webhook — xenditWebhook returns HTTP 501 if unset
XENDIT_CALLBACK_TOKEN=

# AI fallback keys (settings/ai Firestore fields take precedence)
GEMINI_API_KEY=
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
```

Deploy Commands

```
# Build the docs sites
cd docs-site/manualbook && npm run build && cd ../..
cd docs-site/documentation && npm run build && cd ../..

# Build the admin web app
cd apps/admin && flutter build web --release && cd ../..

# Deploy everything
firebase deploy --only functions,firestore:rules,storage:rules,hosting

# Or scope to just the docs sites
firebase deploy --only hosting:manualbook,hosting:documentation
```

Pre-Deploy Checklist

- `flutter analyze` clean across all 4 packages (tk_core, admin, pelanggan, kru).
- `flutter test packages/tk_core apps/admin` — all green.
- `cd firebase/functions && npx tsc -p .` — no type errors.
- Firestore Security Rules reviewed for any collection touched this release.
- New Cloud Functions match the region convention (`asia-southeast2`).

Previous
Notifications

Next
Changelog & Roadmap

OPERATIONS

Changelog & Roadmap

Development phases, from the initial thesis-scoped MVP through the current business-layer feature set.

FASE 1

Bug Blockers

- Splash/launcher fixes
- P6 billing blank-screen fix
- Voucher rules deploy
- Multi-crew assignment wiring
- Profile photos
- Payment proof + QRIS display
- Dashboard status chart
- System bar consistency audit

FASE 2

Customer UX

- Wizard-style booking form
- Saved addresses + map picker

- Reverse-geocoding (Nominatim)
- Full catalog with category filter
- Active-order home card
- Cancel + CS contact from tracking
- Granular permissions + Terms
- Chat + call wiring

FASE 3

Crew (Kru)

- FCM push + assignment reminders (cron)
- Real-road OSRM routing
- Custom notification sound

FASE 4

Admin

- Full dynamic pricing catalog editor
- Crew lifecycle (keahlian/tipe/status)
- Skill-based assignment matching
- New-user + per-phone voucher anti-abuse
- Service area module
- TOTP 2FA
- Admin team management (buatAdmin/setNonaktifAdmin)

FASE 5

Business Systems

- Commission config (global + per-service)
- Cash deposit ledger (kasKru/setoran)
- Referral program with anti-abuse
- Modular payment gateway (Static + Xendit stub)
- WhatsApp notifications (Fonnte)

FASE 6

Quality & Release

- Maintenance mode / forced update (settings/app)
- Skeleton loading states
- Memory-leak audit (chat screen controllers)
- Extended black-box test coverage

POST-FASE 6

Business Expansion

- Multi-provider AI (Gemini/OpenAI/Anthropic) — CS chat (csAi) + business analyst (analisaBisnisAi)
- Xendit dynamic invoicing activated (buatTagihanXendit + xenditWebhook)
- Capacity-based booking (replaces boolean slot lock with a live crew-count counter)
- AI0 Pantau Operasional — live crew map + chat quality monitoring

- A9 Kelola Klien — customer roster with spend aggregation
- Broadcast push notifications + banner promo content
- Auto-cancel unassigned orders after 24h
- Server-authoritative review rating aggregation

For the exact commit history, see the repository's Git log — each feature phase was developed on its own branch and merged via pull request into `dev`.